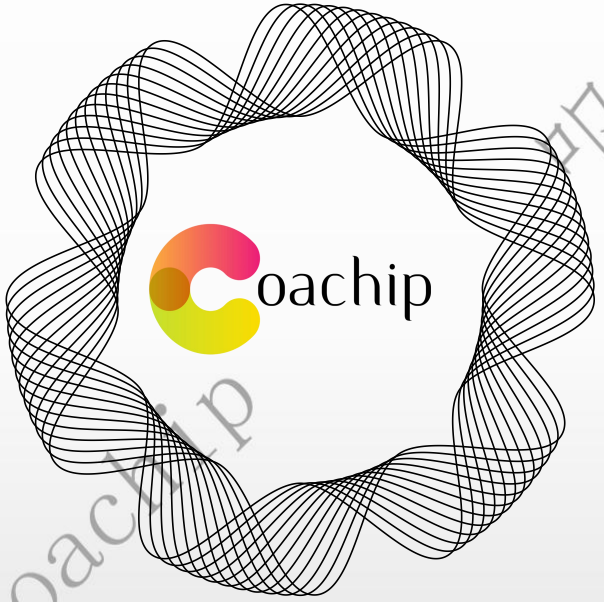


SV的数据类型

叩特电子Coachip

课程内容及目标

课程内容：

- 内建数据类型
- 数组与队列
- 数组的基本操作
- 结构体/枚举类型/字符串

目标：

- 理解内建数据结构
- 了解常用数据类型
- 掌握数组各种操作



叩特电子Coachip

内建数据类型----SV和Verilog数据类型的区别

- 在Verilog中，数据类型分为两类：variable型和net型
  - variable型：reg，integer，time
  - net型：wire，wor，wand等

  在Verilog中，无论是variable型还是net型，都是四值逻辑：0，1，X，Z

- 在System Verilog中，数据类型有两个属性：type和data type，
  - type表明该数据是variable型还是net型;
  - data type表明数据是4值逻辑和2值逻辑;

  variable型数据可以是4值逻辑也可以是2值逻辑

  net型只能是4值逻辑;

叩特电子Coachip

内建数据类型----SV和Verilog数据类型的区别（续）

寄存器（register）类型reg， 线网（net）类型，SV新数据类型logic，它们的区别和联系：

- Verilog作为硬件描述语言，倾向于设计人员自身懂得所描述的电路中哪些变量应该被实现为寄存器，而哪些变量应该被实现为线网类型。这不但有利于后端综合工具，也更便于阅读和理解。
- SV作为侧重于验证的语言，并不十分关切logic对应的逻辑应该被综合为寄存器还是线网，因为logic被使用的场景如果是验证环境，那么它只会作为单纯的变量进行赋值操作，而这些变量也只属于软件环境构建。

logic被推出的另外一个原因也是为了方便验证人员驱动和连接硬件模块、而省去考虑究竟该使用reg还是wire的精力。这既节省了时间，也避免了出错的可能。

System Verilog内建数据类型

logic类型 bit类型

logic相对应的是bit类型，它们均可以构建矢量类型（vector），而它们的区别在于：

logic为四值逻辑，即可以表示**0、1、X、Z**。
bit为二值逻辑，只可以表示**0和1**。

思考：为什么要引入二值逻辑?

叩特电子Coachip

System Verilog内建数据类型（续）

将四值逻辑的类型和二值逻辑的类型分别摘列出来：

四值逻辑类型：integer、reg、logic、net-type（例如wire、tri）

二值逻辑类型：byte、shortint、int、longint、bit

如果按照有符号和无符号的类型进行划分，那么可以将常见的变量类型划分为：

有符号类型：byte、shortint、int、longint、integer

无符号类型：bit、logic、reg、net-type（例如wire、tri）

叩特电子Coachip

System Verilog内建数据类型（续）

通过logic和bit来声明的矢量均为无符号（unsigned）变量，比如：

```
logic [7:0] logic_vector = 8'b1000_0000;
bit [7:0] bit_vector = 8'b1000_0000;
byte signed_vector = 8'b1000_0000;
```

```
initial begin
    $display(“logic_vector= %d ”,logic_vector);
    $display(“bit_vactor = %d ”,bit_vector);
    $display(“signed_vector= %d ”,signed_vector);
end
```

仿真结果如下：

```
logic_vector = 128;
bit_vector = 128;
signed_vector = - 128;
```

叩特电子Coachip

System Verilog内建数据类型（续）

遇到常见的这些变量类型时，应该注意它们的逻辑类型和符号类型，因为在变量运算中，应该避免两种不一致的变量进行操作导致意外的错误。

譬如有符号变量和无符号变量混用的运算结果会出乎意料：

```
byte signed_vector = 8'b1000_0000;
bit [8:0] result_vector;

initial begin
   result_vector = signed_vector
    $display("@1 result_vector= 'h%x ",result_vector);
    result_vector = unsigned'(signed_vector);
   $display("@2 result_vector= 'h%x ",result_vector);
end
```

仿真结果如下：

@1 result_vector = 'h180

@2 result_vector = 'h080

叩特电子Coachip

System Verilog的数据类型 – 内建数据类型（续）

- bit — 1位2值的整数
- byte — 8位2值整数,类似C语言中char
- shortint — 16位2值整数, 类似C语言中short
- int — 32位2值整数, 类似C语言中int
- longint — 64位2值整数, 类似C语言中long long

```
1 bit  resetN;  // a 1-bit wide 2-state variable
2 bit  [63:0] data;  // a 64-bit 2-state variable
3 bit  [0:7]  array [0:255]; // an array of 8-bit 2-state variables
```

在System Verilog中，使用关键字**logic**来代替reg，logic单独使用时，表示的是4值类型数据。如：

```
1 logic  resetN;  // a 1-bit wide 4-state variable
2 logic  [63:0] data;  // a 64-bit wide variable
3 logic  [0:7]  array [0:255]; // an array of 8-bit variables
```

System Verilog内建数据类型（续）

logic类型和bit类型在进行运算时，如果位宽不一致会发生什么？

```
logic [3:0] x_vector = 4'b111x;
bit [2:0]    bit_vector;

initial begin
   $display("@1 x_vector= 'b%b ",x_vector);
    bit_vector = x_vector;
   $display("@2 bit_vector= 'b%b",bit_vector);
end
```

仿真结果如下：

@1 x_vector = 'b111x

@2 bit_vector = 'b110

Coach confidential

System Verilog内建数据类型（续）

- 在不同数据类型进行操作时应该注意变量的：
  - 逻辑数值类型
  - 符号类型
  - 矢量位宽

叩特电子Coachip

System verilog的数据类型 – 内容回顾

思考：为什么要引入二值逻辑?

- **内建数据类型**

  将四值逻辑的类型和二值逻辑的类型分别摘列出来：

  四值逻辑类型：integer、reg、logic、net-type（例如wire、tri）

  二值逻辑类型：byte、shortint、int、longint、bit

  如果按照有符号和无符号的类型进行划分，那么可以将常见的变量类型划分为：

  有符号类型：byte、shortint、int、longint、integer

  无符号类型：bit、logic、reg、net-type（例如wire、tri）

叩特电子Coachip

练习一

```
logic [7:0] logic_vector = 8'b1000_0000;
bit     [7:0] bit_vector    = 8'b1000_0000;
byte      signed_vector = 8'b1000_0000;

initial begin
   $display(“logic_vector= %d ”,logic_vector);
   $display(“bit_vactor = %d ”,bit_vector);
   $display(“signed_vector= %d ”,signed_vector);
end
```

看运行结果是否和预期一致

叩特电子Coachip

练习二

```
byte signed_vector = 8'b1000_0000;
 bit [8:0] result_vector;

initial begin
   result_vector = signed_vector;
    $display(“@1 result_vector= 'h%x ”,result_vector);
    result_vector = unsigned'(signed_vector);
   $display(“@2 result_vector= 'h%x ”,result_vector);
end
```

看运行结果是否和预期一致

叩特电子Coachip

练习三

```
logic [3:0] x_vector = 4'b111x;
bit     [2:0]    bit_vector;

initial begin
   $display(“@1 x_vector= 'b%b ”,x_vector);
    bit_vector = x_vector;
   $display(“@2 bit_vector= 'b%b”,bit_vector);
end
```

看运行结果是否和预期一致

叩特电子Coachip

Q&A

昂特电子Coachip